

若需在 View 直接注入服務，比較正確的應用是，該服務屬於前端服務，專為前端網頁產生 UI 介面資料，而不涉及中後端服務或商業邏輯，此時在 View 中直接注入服務就有其正當性。

範例 13-2 將 IZipCodeService 服務注入 View，用以查詢郵遞區號

以下建立一個查詢台灣縣市的 Zip Code 服務，然後直接注入到 View，供其查詢郵遞區號，請參考 ViewInjection 控制器及 InjectZipcodeService() 方法。



圖 13-3 在 View 注入郵遞區號服務

step 01 建立 ZipcodeViewModel 模型，用來持有縣市郵遞區號資料

 ViewModels/ZipcodeViewModel.cs

```
namespace Mvc7_DependencyInjection.ViewModels
{
    public class ZipcodeViewModel
    {
        public string CityId { get; set; }
        public string CityName { get; set; }
        public List<District> Districts { get; set; }
    }

    public class District
    {
        public string Id { get; set; }
        public string Name { get; set; }
        public string Zipcode { get; set; }
    }
}
```

```

    }
}

```

step 02 建立 IZipCodeService 抽象介面



Interfaces/IZipcodeService.cs

```

public interface IZipcodeService
{
    string Caption { get; }
    List<ZipcodeViewModel> Cities { get; set; }
    string QueryZipcode(string cityName, string districtName)
}

```

step 03 TaiwanZipcodeService 繼承並實作 IZipCodeService 介面。但因全台灣縣市 Zip Code 資料過多，為節省時間，僅建立基隆、台北市、新北市及桃園市的郵遞區號資料



Services/TaiwanZipcodeService.cs

```

public class TaiwanZipcodeService : IZipcodeService
{
    public string Caption { get; } = "Zipcode";
    public List<ZipcodeViewModel> Cities { get; set; }
    public TaiwanZipcodeService()
    {
        Cities = new List<ZipcodeViewModel>
        {
            new ZipcodeViewModel {
                CityId = "01",
                CityName = "基隆市",
                Districts = new List<District>
                {
                    new District { Id = "01", Name = "仁愛區", Zipcode = "200" },
                    ...
                }
            },
            new ZipcodeViewModel {
                CityId = "02",
                CityName = "台北市",
                Districts = new List<District>
                {

```

```
        new District { Id = "01", Name = "中正區", Zipcode = "100" },
        ...
    },
    new ZipcodeViewModel {
        CityId = "03",
        CityName = "新北市",
        Districts = new List<District>
        {
            new District { Id = "01", Name = "萬里區", Zipcode = "207" },
            ...
        }
    },
    new ZipcodeViewModel {
        CityId = "04",
        CityName = "桃園市",
        Districts = new List<District>
        {
            new District { Id = "01", Name = "中壢區", Zipcode = "320" },
            ...
        }
    }
};

//將 List<ZipcodeViewModel> 集合轉成 JSON 文字格式
//string json = Newtonsoft.Json.JsonConvert.SerializeObject(Cities);
}

//查詢郵遞區號
public string QueryZipcode(string cityName, string districtName)
{
    if (string.IsNullOrEmpty(cityName) &&
        string.IsNullOrEmpty(districtName))
    {
        return "Error : Must provide the City & District Name";
    }

    //1.查詢 City
    var _city = Cities.FirstOrDefault(c => c.CityName.Contains(cityName));

    if (_city == null)
    {
        return "查無此 City";
    }
}
```

```

//2.查詢 District

var _district = _city.Districts
    .FirstOrDefault(d => d.Name.Contains(districtName));

if (_district is null)
{
    return "查無此 District";
}

//3.讀取 Zipcode
string _zipcode = _district.Zipcode;

//或將三行查詢成合一行(此查詢沒有做防呆,可能產生 Null Exception)
string zipc = Cities
    .FirstOrDefault(c => c.CityName.Contains(cityName)).Districts
    .FirstOrDefault(d => d.Name.Contains(districtName)).Zipcode;

return _zipcode;
}
}

```

step 04 在 Program.cs 中以 AddSingleton 方法註冊 IZipcodeService 相依性

 Program.cs

```
pubbuilder.Services.AddSingleton<IZipcodeService, TaiwanZipcodeService>();
```

郵遞區號介面

台灣郵遞區號實作

step 05 在 View 中以 @inject 指示詞注入 IZipcodeService 相依性物件，將縣市和區的名稱傳入 QueryZipcode 方法中查詢郵遞區號

 Views/ViewInjection/InjectZipcodeService.cshtml

```

@inject IZipcodeService zipcodeService ← 注入相依性物件
<ul>
    <li>基隆市,七堵區 Zip code : @zipcodeService.QueryZipcode("基隆市", "七堵區")</li>
    <li>台北市,信義區 Zip code : @zipcodeService.QueryZipcode("台北市", "信義區")</li>
    <li>新北市,板橋區 Zip code : @zipcodeService.QueryZipcode("新北市", "板橋區")</li>
    <li>桃園市,八德區 Zip code : @zipcodeService.QueryZipcode("桃園市", "八德區")</li>

```